Lab 7: Regional frequency analysis with a CO2 trend

2026-10-09

In this lab, you will estimate the 100-year daily rainfall in 2026 at a coastal gauge with a short record. You will let the rainfall extremes change with CO2, and borrow data from nearby gauges to make up for the short record. **Class does not meet** on Friday, October 9, so you will work through this lab on your own.

Objectives

By the end of this lab, you will be able to:

1. Fit a GEV that changes with CO2 by weighted likelihood.
2. Choose which stations to borrow from with a weight function.
3. Get an interval by MCMC and by a bootstrap over years, and explain why they differ.
4. Make and defend a choice of weight function and of which parameters to share.

Setup

First, load the packages.

```
using Pkg
here = dirname(@__FILE__)
Pkg.activate(here)
Pkg.instantiate()

using CairoMakie, CSV, DataFrames, Dates, Distributions, LaTeXStrings,
      NaturalEarth, Turing
using Printf, Random, Statistics

Random.seed!(543)
set_theme!(theme_minimal(); fontsize=14)
colors = Makie.wong_colors()
```

Rainfall data

You will analyze annual maximum daily rainfall from NOAA Atlas 14, Volume 11, which covers Texas [1]. The file `dur01d_ams_na14v11.txt` is the 1-day annual maximum series that NOAA published with the atlas, downloaded unchanged from the NOAA Hydrometeorological Design Studies Center. It holds 816 stations, with one annual maximum per calendar year, in inches, through 2017. After a title line, the file has one block per station, separated by blank lines. Each block starts with a header line giving the station's ID, name, state, latitude, longitude, and elevation in feet, followed by one line per year giving the date of that year's maximum and its value.

i Code Folds

Some of the code blocks in this lab are "folded" to improve readability. Click the triangle next to a label such as "Show the file reader" to see the code. A numbered marker at the end of a line explains that line; hover over it to read the note.

```
function read_station(block)
    header, rows... = split(block, '\n')
    fields = strip.(split(header, ','))
    lat, lon, elev_ft = parse.(Float64, fields[(end - 2):end])
    days = split.(rows)
    return DataFrame(;
                     id=fields[1], name=join(fields[2:(end - 4)], ","), lat,
                     lon, elev_ft,
                     date=Date.(first.(days), dateformat"mm/dd/yyyy"),
                     prcp_in=parse.(Float64, last.(days)))
end

function read_atlas14(path)
    blocks = split(strip(join(Iterators.drop(eachline(path), 1), '\n')),
                   r"\n\s*\n")
    return reduce(vcat, read_station.(blocks))
end

ann_max_texas = read_atlas14(joinpath(here, "dur01d_ams_na14v11.txt"))
ann_max_texas.year = year.(ann_max_texas.date)
first(ann_max_texas, 4)
```

Line 2

The first line of a block is the station's header, and each line after it is one year.

Line 4

A station's name can contain a comma, so the header is read from both ends: the ID is the first field, and the last three are latitude, longitude, and elevation.

Line 15

Skip the title line and split the rest of the file into one block per station.

Line 16

Turn each block into a table and stack the tables, one row per station and year.

```
4×8 DataFrame
 Row │ id          name                        lat      lon       elev_ft  date  ⋯
     │ SubStrin…   String                      Float64  Float64   Float64  Date  ⋯
─────┼──────────────────────────────────────────────────────────────────────────────
   1 │ 60-0011     CLEAR CK AT BAY AREA BLVD   29.4977  -95.1599      2.0  1987- ⋯
```

```
   2 | 60-0011    CLEAR CK AT BAY AREA BLVD  29.4977  -95.1599      2.0  1988-
   3 | 60-0011    CLEAR CK AT BAY AREA BLVD  29.4977  -95.1599      2.0  1989-
   4 | 60-0011    CLEAR CK AT BAY AREA BLVD  29.4977  -95.1599      2.0  1990-
                                                              3 columns omitted
```

CO2 data

The file `logCO2.csv` has one row per year from 1849 to 2025. Its column `log_CO2` is the natural log of annual mean atmospheric CO2, in parts per million. Before 1959 it comes from the Law Dome ice core record [2], [3]; from 1959 it is the Mauna Loa annual mean from NOAA's Global Monitoring Laboratory.

The covariate is the number of doublings of CO2 above the pre-industrial level of 280 ppm:

$$x_t = \log_2 \frac{\mathrm{CO}_2(t)}{280}.$$

The file stops at 2025, at 427 ppm. CO2 rose about 2.6 ppm per year from 2015 to 2025, so for 2026 we assume 430 ppm.

```
co2 = CSV.read(joinpath(here, "logCO2.csv"), DataFrame)
co2.x = (co2.log_CO2 .- log(280)) ./ log(2)
x_2026 = log2(430 / 280)
x_1950 = only(co2.x[co2.year .== 1950])

ann_max_texas = innerjoin(ann_max_texas, co2; on=:year)
```

The target station

You will estimate the 100-year daily rainfall in 2026 at Galveston Scholes Field (station 79-0047), on Galveston Island. Many real gauges have short records, so you will use only its 21 years from 1997 to 2017.

```
filter!(row -> row.id != "79-0047" || row.year >= 1997, ann_max_texas)
scholes_field = first(ann_max_texas[ann_max_texas.id .== "79-0047", :])
```

Choosing the stations

To borrow from neighbors, we first need each station's distance to Scholes Field. With latitudes ϕ_1, ϕ_2 and longitudes λ_1, λ_2, the haversine formula gives the great-circle distance

$$d = 2R \arcsin \sqrt{\sin^2\left(\frac{\phi_2 - \phi_1}{2}\right) + \cos\phi_1 \cos\phi_2 \sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right)},$$

where $R = 3958.8$ miles is the radius of the Earth.

```
function miles_between(lat1, lon1, lat2, lon2)
    R = 3958.8
    north_south = sind((lat2 - lat1) / 2)^2
    east_west = cosd(lat1) * cosd(lat2) * sind((lon2 - lon1) / 2)^2
```

```
    return 2R * asin(sqrt(north_south + east_west))
end

ann_max_texas.miles = miles_between.(scholes_field.lat, scholes_field.lon,
                                     ann_max_texas.lat, ann_max_texas.lon)
```

The weight function

A weight function says how much each station counts, from 1 (as much as Scholes Field) to 0 (not at all). As an example, we consider one that falls linearly with distance d and reaches zero at a radius r:

$$w(d) = \max\left(0, 1 - \frac{d}{r}\right).$$

```
radius = 50
weight(miles) = max(0.0, 1 - miles / radius)
```

To try a different weight function, edit `weight(miles)` above and rerun the notebook from there. Some options:

- **Region of influence:** $w(d) = 1$ if $d \le r$, and 0 otherwise.
- **Inverse distance squared:** $w(d) = \min(1, (d_0/d)^2)$ if $d \le r$, and 0 otherwise.
- **Exponential kernel:** $w(d) = \exp(-d/L)$, with length scale L.

We keep the stations with weight above zero. The effective number of stations, $n_{\text{eff}} = \sum_s w_s$, is how many stations' worth of data the fit uses.

```
ann_max_texas.w = weight.(ann_max_texas.miles)
ann_max_region = ann_max_texas[ann_max_texas.w .> 0, :]
stations = combine(groupby(ann_max_region, [:id, :name, :lat, :lon, :miles, :w]),
                   nrow => :n_years)
sort!(stations, :miles)
@printf("%d stations kept, n_eff = %.1f\n", nrow(stations), sum(stations.w))
```

```
20 stations kept, n_eff = 6.1
```

The maps in this lab draw land from the 1:10m layer of Natural Earth, which the `NaturalEarth` package downloads on first use. `map_axis` makes an axis with that background.

```
land = naturalearth("land", 10)

function map_axis(position)
    ax = Axis(position; autolimitaspect=1 / cosd(scholes_field.lat),
              backgroundcolor=:aliceblue)
    poly!(ax, land.geometry; color=:gray95, strokecolor=:gray60,
          strokewidth=0.5, xautolimits=false, yautolimits=false)
```

```
    hidedecorations!(ax)
    return ax
end
```

Line 4

Stretch latitude by $1/\cos\phi$ at Scholes Field, so a mile east and a mile north are the same length on the map.

```
let
    fig = Figure(; size=(800, 380))
    ax_map = map_axis(fig[1, 1])
    stations_plot = scatter!(ax_map, stations.lon, stations.lat;
                             color=stations.w, colormap=:viridis,
                             colorrange=(0, 1), markersize=10)
    scatter!(ax_map, [scholes_field.lon], [scholes_field.lat]; marker=:star5,
             color=:black, markersize=20)
    Colorbar(fig[1, 2], stations_plot; label="weight")

    at_scholes = ann_max_region.id .== scholes_field.id
    ax_series = Axis(fig[1, 3]; xlabel="year", ylabel="annual maximum (inches)")
    scatter!(ax_series, ann_max_region.year, ann_max_region.prcp_in;
             color=(colors[1], 0.4), markersize=5)
    scatter!(ax_series, ann_max_region.year[at_scholes],
             ann_max_region.prcp_in[at_scholes]; color=:black, markersize=8)
    fig
end
```

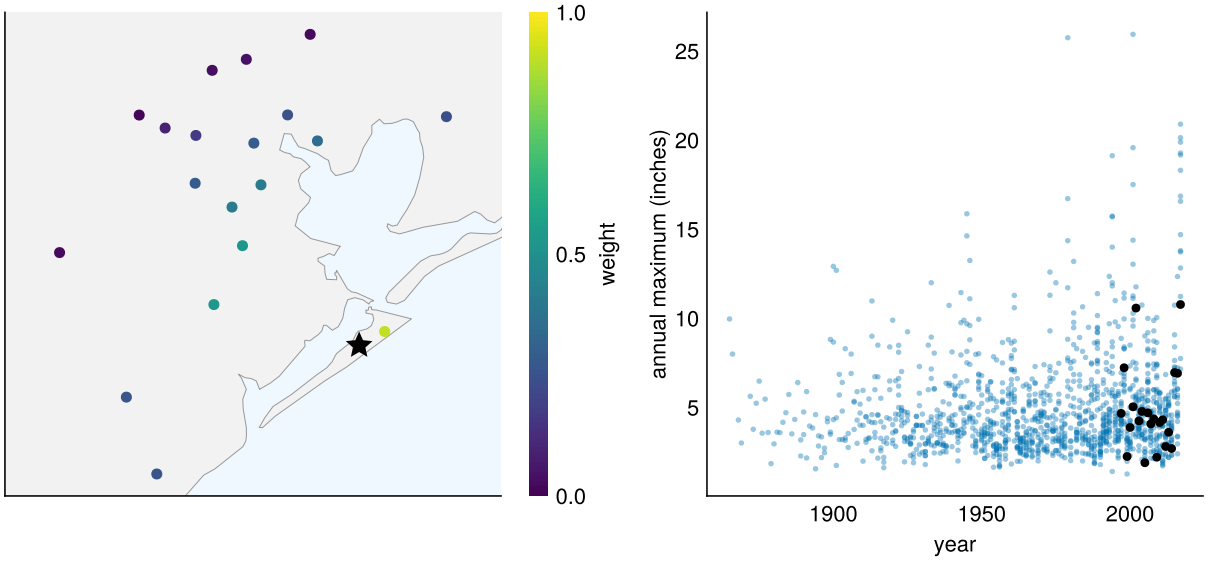


Figure 1: Left: the kept stations, colored by weight, with Galveston Scholes Field as the star. Right: their annual maxima, with Scholes Field in black.

Nonstationary model

We allow a trend in both the location and the scale, and assume the two are proportional:

$$
\begin{aligned}
y_t &\sim \mathrm{GEV}(\mu_t, \sigma_t, \xi) \\
\mu_t &= \mu_0 e^{\beta x_t} \\
\sigma_t &= \kappa \mu_t
\end{aligned}
$$

In lab 6, the location and the log scale each had their own trend. Here a single parameter, β, sets the trend in both:

- μ_0 is the location at the pre-industrial level, $x = 0$.
- When x rises by Δx, both μ_t and σ_t are multiplied by $e^{\beta \Delta x}$.
- κ is the ratio of scale to location, the same in every year.
- ξ is the shape, also the same in every year.

Since the location and scale change by the same factor and the shape does not change, every return level changes by that factor too. We report the trend as the percent change in every return level from 1950 to 2026, $100\left(e^{\beta(x_{2026} - x_{1950})} - 1\right)$.

To borrow from neighbors, we weight each station's log-likelihood by w_s and add them up:

$$
\ell(\mu_0, \kappa, \beta, \xi) = \sum_s w_s \sum_t \log f(y_{s,t} \mid \mu_0, \kappa, \beta, \xi),
$$

where f is the GEV density. With a single station and $w = 1$, this is the ordinary log-likelihood.

`weighted_gev` writes this as a Turing model.

```
@model function weighted_gev(y, x, w)
    μ0 ~ LogNormal(1.5, 0.5)
    κ ~ LogNormal(-1, 0.5)
    β ~ Normal(0, 0.5)
    ξ ~ Uniform(-0.5, 0.5)
    μ = μ0 .* exp.(β .* x)
    σ = κ .* μ
    if !all(σ .> 0)
        Turing.@addlogprob! -Inf
        return nothing
    end
    Turing.@addlogprob! sum(w .* logpdf.(GeneralizedExtremeValue.(μ, σ, ξ), y))
end
```

Lines 2-5

These priors give 2026 100-year levels between about 3 and 70 inches, and trends between about -37% and $+58\%$ from 1950 to 2026.

Line 8

A large optimizer step can push μ_t so close to zero or so large that the scale rounds to 0 or becomes `NaN`; reject that step rather than stop with an error.

Line 12

A `y ~` line cannot weight observations, so the model adds the weighted log-likelihood itself.

`weighted_gevfit` finds the parameters with the highest posterior density, using Turing's `maximum_a_posteriori`. Two helper functions after it keep the later code short.

```
function weighted_gevfit(y, x, w)
    start = InitFromParams((μ0=median(y), κ=std(y) / median(y), β=0.0, ξ=0.1))
    fit = maximum_a_posteriori(weighted_gev(y, x, w); initial_params=start)
    if string(fit.optim_result.retcode) != "Success"
        @warn "weighted_gevfit did not converge" fit.optim_result.retcode
    end
    return NamedTuple(fit.params)
end

function level_2026(fit)
    μ = fit.μ0 * exp(fit.β * x_2026)
    return quantile(GeneralizedExtremeValue(μ, fit.κ * μ, fit.ξ), 0.99)
end
trend_percent(fit) = 100 * (exp(fit.β * (x_2026 - x_1950)) - 1)
```

Line 2

Start the optimizer from the data's median and spread, with no trend.

Line 4

Warn if the optimizer fails, since the parameters it returns may then not be the best ones.

Line 10

The fitted 100-year level in 2026.

Line 14

The trend, as the percent change in every return level from 1950 to 2026.

No pooling

First, we fit each kept station to its own record.

```
stations.fit = map(stations.id) do id
    rows = ann_max_region[ann_max_region.id .== id, :]
    return weighted_gevfit(rows.prcp_in, rows.x, ones(nrow(rows)))
end
stations.κ = [fit.κ for fit in stations.fit]
stations.trend_percent = trend_percent.(stations.fit)
stations.level_2026 = level_2026.(stations.fit)
stations[:, [:name, :miles, :n_years, :κ, :trend_percent, :level_2026]]
```

```
20×6 DataFrame
 Row │ name                          miles     n_years  κ         trend_perce ⋯
```

	String	Float64	Int64	Float64	Float64	…
1	GALVESTON SCHOLES FLD	0.0	21	0.406968	2.723	…
2	GALVESTON	4.52325	137	0.409822	-15.285	
3	ALVIN	23.4338	107	0.358851	6.035	
4	CLEAR CK AT BAY AREA BLVD	23.8355	30	0.363389	4.021	
5	TURKEY CK AT FM 1959	29.176	30	0.283562	16.138	…
6	ARMAND BYU AT GENOARED BLF RD	29.2355	31	0.348342	18.577	
7	GOOSE CREEK	32.434	76	0.349925	-2.708	
8	HOUSTON DEER PARK	35.4218	63	0.376562	-6.656	
⋮	⋮	⋮	⋮	⋮	⋮	
14	HOUSTON-PORT	41.2936	29	0.393411	9.131	…
15	HOUSTON WB CITY	45.2242	125	0.358468	19.718	
16	HOUSTON SAN JACINTO DA	47.765	64	0.382318	17.808	
17	GREENS BYU AT MT HOUSTON PKWY	48.4303	31	0.414977	7.869	
18	THOMPSONS 3 WSW	48.6821	59	0.37095	0.204	…
19	CEDAR BYU AT US 90	48.9298	31	0.453029	2.987	
20	HOUSTON HEIGHTS	49.426	57	0.280484	88.802	

2 columns and 5 rows omitted

```
let
    fig = Figure(; size=(900, 380))
    biggest = maximum(abs, stations.trend_percent)

    ax_trend = map_axis(fig[1, 1])
    trend_plot = scatter!(ax_trend, stations.lon, stations.lat;
                          color=stations.trend_percent, colormap=:BrBG,
                          colorrange=(-biggest, biggest), markersize=12,
                          strokewidth=0.5, strokecolor=:gray30)
    Colorbar(fig[1, 2], trend_plot; label="trend, 1950 to 2026 (%)")

    ax_level = map_axis(fig[1, 3])
    level_plot = scatter!(ax_level, stations.lon, stations.lat;
                          color=stations.level_2026, colormap=:viridis,
                          markersize=12, strokewidth=0.5, strokecolor=:gray30)
    Colorbar(fig[1, 4], level_plot; label="2026 100-year level (inches)")

    for ax in (ax_trend, ax_level)
        scatter!(ax, [scholes_field.lon], [scholes_field.lat]; marker=:star5,
                 color=:black, markersize=20)
    end
    fig
end
```

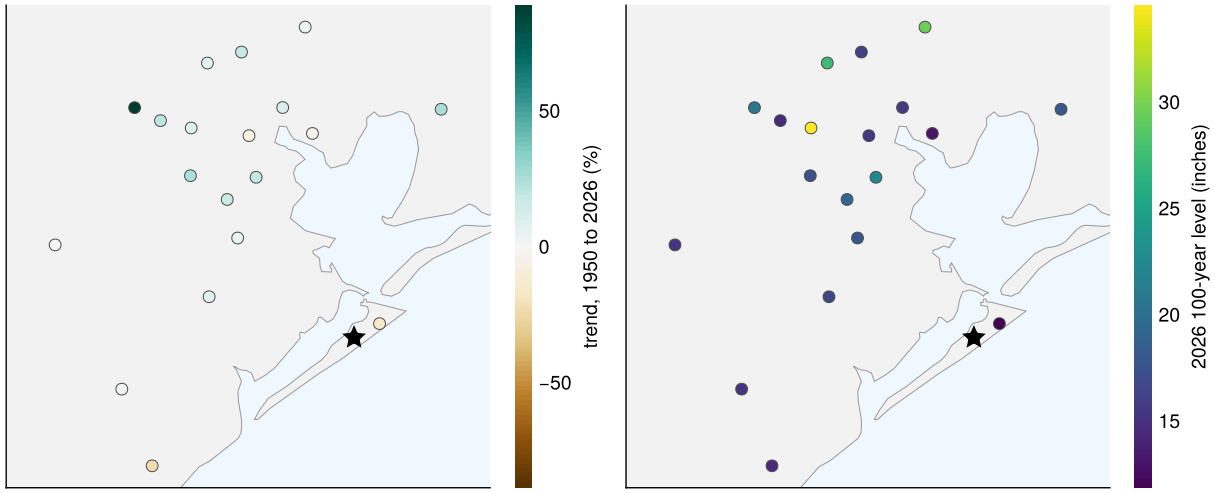


Figure 2: Each station fit to its own record. Left: trend, as the percent change in every return level from 1950 to 2026. Right: 100-year daily rainfall in 2026. Scholes Field is the star.

Your answer: How much do the trends differ between stations a few miles apart? Is that more likely a real difference in climate or noise from short records, and what in the table supports your answer?

Galveston Scholes Field and GALVESTON are only 4.52 miles apart, but their fitted trends are +2.72% and −15.29%, respectively—a difference of about 18 percentage points. This difference is more likely due to sampling variability and differences in record coverage than a sharp local difference in climate. Scholes Field has only 21 years of observations, compared with 137 years at GALVESTON. However, the table alone cannot confirm this explanation. Comparing both stations over the same period and examining uncertainty intervals for their trends would help assess whether the difference reflects sampling variability.

Your answer: How much does x_t change over Scholes Field's 21 years, compared with a record that starts around 1900? So what sets Scholes Field's trend when you fit it alone?

The x_t increases 0.161 from 1997 to 2017, and increases 0.463 from 1900 to 2017. Scholes Field's short record and small change in x_t provide limited information about the trend. When fitted alone, the trend estimate is strongly affected by random variability in the short record.

```
x_at(yr) = only(co2.x[co2.year .== yr])


Δx_short = x_at(2017) - x_at(1997)


Δx_long = x_at(2017) - x_at(1900)

@printf("Scholes Field Δx: %.3f\n", Δx_short)
@printf("1900–2017 Δx: %.3f\n", Δx_long)
@printf("Long-record change is %.2f times larger\n", Δx_long / Δx_short)
```

```
Scholes Field Δx: 0.161
1900–2017 Δx: 0.463
Long-record change is 2.88 times larger
```

Borrowing from neighbors

Now we fit Scholes Field with every kept station's data, weighted by `weight`.

```
pooled = weighted_gevfit(ann_max_region.prcp_in, ann_max_region.x,
                         ann_max_region.w)
scholes_alone = only(stations.fit[stations.id .== scholes_field.id])
for (label, fit) in ("alone" => scholes_alone, "pooled" => pooled)
    @printf("%-6s trend %+.0f%% from 1950 to 2026, 2026 100-year level %.1f inches\n",
            label, trend_percent(fit), level_2026(fit))
end
```

```
alone  trend +3% from 1950 to 2026, 2026 100-year level 13.9 inches
pooled trend +10% from 1950 to 2026, 2026 100-year level 16.2 inches
```

Uncertainty

A point estimate alone does not say how much to trust it. We get an interval for the pooled fit in two ways.

MCMC

MCMC samples the posterior of the same weighted model.

```
chain = sample(weighted_gev(ann_max_region.prcp_in, ann_max_region.x,
                            ann_max_region.w), NUTS(), 1000; progress=false)
mcmc_fits = [(μ0=a, κ=b, β=c, ξ=d)
             for (a, b, c, d) in zip(vec(chain[:μ0]), vec(chain[:κ]),
                                     vec(chain[:β]), vec(chain[:ξ]))]
```

Bootstrap over years

The bootstrap refits the model to resampled data, drawing whole years rather than single values.

```
years = unique(ann_max_region.year)

function bootstrap_fit()
    picked = rand(years, length(years))
    rows = reduce(vcat, [ann_max_region[ann_max_region.year .== yr, :]
                         for yr in picked])
    return weighted_gevfit(rows.prcp_in, rows.x, rows.w)
end

bootstrap_fits = [bootstrap_fit() for _ in 1:500]
```

Line 4

Draw as many years as the record has, with replacement.

Line 5

Keep every station's value from each drawn year, so a storm that hit many stations in one year stays together.

```
let
    fig = Figure(; size=(800, 330))
    ax_trend = Axis(fig[1, 1]; xlabel="trend, 1950 to 2026 (%)")
    ax_level = Axis(fig[1, 2]; xlabel="2026 100-year level (inches)")
    for (i, (label, fits)) in enumerate(("MCMC" => mcmc_fits,
                                         "bootstrap" => bootstrap_fits))
        hist!(ax_trend, trend_percent.(fits); bins=40, normalization=:pdf,
              color=(colors[i], 0.5), label=label)
        hist!(ax_level, level_2026.(fits); bins=40, normalization=:pdf,
              color=(colors[i], 0.5), label=label)
    end
    hideydecorations!.((ax_trend, ax_level))
    axislegend(ax_level)
    fig
end
```

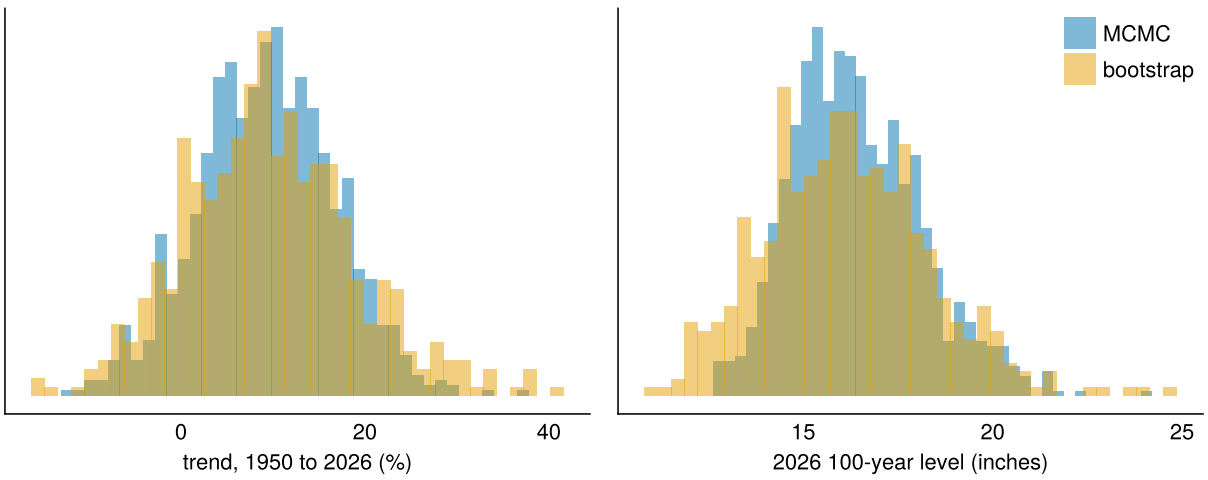


Figure 3: The pooled fit at Scholes Field by MCMC and by the bootstrap over years. Left: trend. Right: 2026 100-year level.

```
for (label, fits) in ("MCMC" => mcmc_fits, "bootstrap" => bootstrap_fits)
    levels = level_2026.(fits)
    @printf("%-9s 2026 100-year level, 95%% interval %.1f to %.1f inches\n",
            label, quantile(levels, 0.025), quantile(levels, 0.975))
end
```

```
MCMC      2026 100-year level, 95% interval 13.5 to 20.2 inches
bootstrap 2026 100-year level, 95% interval 12.2 to 20.6 inches
```

Your answer: How many kept stations have a 2017 annual maximum, and on what dates did those maxima fall? The weighted likelihood treats every station's value as independent. Which method keeps the 2017 values together, and which interval would you expect to be wider?

17 kept station have a 2017 annual maximum: 8 on August 26, 6 on August 27, 2 on August 28, and 1 on August 29. The clustering of dates suggests that these observations likely reflect a shared storm event and are therefore dependent. The bootstrap over years keeps all the 2017 values together, preserving their dependence across stations. I would expect its interval to be wider than the MCMC interval.

```
data_2017 = filter(row -> row.year == 2017, ann_max_region)

length(unique(data_2017.id))
combine(groupby(data_2017, :date), nrow => :n_stations)
```

```
4×2 DataFrame
 Row │ date        n_stations
     │ Date        Int64
─────┼──────────────────────
   1 │ 2017-08-26           8
   2 │ 2017-08-27           6
   3 │ 2017-08-28           2
   4 │ 2017-08-29           1
```

Your analysis

Change the weight function

Change `radius` and rerun the notebook from the weight function down. Then code one of the other weight functions and run it the same way.

Your answer: Make a table with one row per weight function you tried, giving the number of stations kept, n_{eff}, the pooled trend, and the 2026 100-year level with its bootstrap interval. Which settings give an estimate dominated by variance, and which by bias? Name a physical reason a wide region could be biased for a station on Galveston Island.

Here, d is distance from Scholes Field in miles. The pooled trend is the percentage change from 1950 to 2026. Rainfall levels and intervals are in inches.

Weight function	Stations kept	n_{eff}	Pooled trend	2026 100-year level (95% bootstrap interval)
Linear: $w(d) = \max(0, 1 - d/25)$	4	1.93	−3.83%	13.68 (10.31–18.12)

Weight function	Stations kept	n_{eff}	Pooled trend	2026 level	100-year (95% bootstrap interval)
Linear: $w(d) = \max(0, 1 - d/50)$	20	6.13	+9.83%	16.24	(12.92–20.93)
Region of influence: $w(d) = 1$ if $d \leq 50$, otherwise 0	20	20.00	+17.79%	17.56	(13.34–22.89)
Inverse squared: $w(0) = 1$; $w(d) = \min(1, (10/d)^2)$ if $0 < d \leq 50$, otherwise 0	20	3.45	+3.76%	15.19	(11.94–19.52)
Exponential: $w(d) = \exp(-d/25)$, no cutoff	816	9.68	+16.28%	16.61	(12.75–22.15)

The 25-mile linear setting (Linear: $w(d) = \max(0, 1 - d/25)$) is more vulnerable to variance because it uses only four stations. The 50-mile region of influence setting (Region of influence: $w(d) = 1$ if $d \leq 50$, otherwise 0) and the exponential setting (Exponential: $w(d) = \exp(-d/25)$) have greater potential for bias because they give more influence to distant stations.

Galveston Island's coastal rainfall is influenced by Gulf moisture and tropical storms, so inland stations may have different rainfall patterns.

```
using CSV, DataFrames, Dates, Distributions, Turing
using Random, Statistics, Printf

let
    lab_dir = "/Users/canliu/PHD-Courses/26Fall/CEVE543/labs/CEVE543-F26-CL333-lab07"
    B = 500
    Random.seed!(543)

    # 1. Read rainfall data
    function read_station(block)
        header, rows... = split(block, '\n')
        fields = strip.(split(header, ','))
        days = split.(rows)

        DataFrame(
            id = fields[1],
            lat = parse(Float64, fields[end - 2]),
```

```
        lon = parse(Float64, fields[end - 1]),
        date = Date.(first.(days), dateformat"mm/dd/yyyy"),
        rainfall = parse.(Float64, last.(days))
    )
end

raw = read(joinpath(lab_dir, "dur01d_ams_na14v11.txt"), String)
raw = replace(raw, "\r\n" => "\n")
body = split(raw, '\n'; limit=2)[2]
blocks = split(strip(body), r"\n\s*\n")
data = reduce(vcat, read_station.(blocks))
data.year = year.(data.date)

# Keep only 1997–2017 for Scholes Field
filter!(r -> r.id != "79-0047" || 1997 <= r.year <= 2017, data)

# 2. Read CO₂ data
co2 = CSV.read(joinpath(lab_dir, "logCO2.csv"), DataFrame)
co2.x = (co2.log_CO2 .- log(280)) ./ log(2)

x2026 = log2(430 / 280)
x1950 = only(co2.x[co2.year .== 1950])

data = innerjoin(data, select(co2, :year, :x); on=:year)

# 3. Calculate distance from Scholes Field
target = first(data[data.id .== "79-0047", :])

function distance_miles(lat1, lon1, lat2, lon2)
    a = sind((lat2 - lat1) / 2)^2 +
        cosd(lat1) * cosd(lat2) * sind((lon2 - lon1) / 2)^2

    2 * 3958.8 * asin(sqrt(clamp(a, 0.0, 1.0)))
end

data.miles = distance_miles.(
    target.lat, target.lon, data.lat, data.lon
)

# 4. Weighted GEV model: all parameters shared
@model function comparison_gev(y, x, w)
    μ0 ~ LogNormal(1.5, 0.5)
    κ ~ LogNormal(-1, 0.5)
    β ~ Normal(0, 0.5)
    ξ ~ Uniform(-0.5, 0.5)
```

```
        μ = μ0 .* exp.(β .* x)
        σ = κ .* μ

        if !all(isfinite.(σ) .& (σ .> 0))
            Turing.@addlogprob! -Inf
            return nothing
        end

        Turing.@addlogprob! sum(
            w .* logpdf.(GeneralizedExtremeValue.(μ, σ, ξ), y)
        )
    end

    function fit_model(y, x, w)
        start = InitFromParams((
            μ0 = median(y),
            κ = std(y) / median(y),
            β = 0.0,
            ξ = 0.1
        ))

        fit = maximum_a_posteriori(
            comparison_gev(y, x, w);
            initial_params=start
        )

        if string(fit.optim_result.retcode) != "Success"
            error("MAP fit did not converge: $(fit.optim_result.retcode)")
        end

        NamedTuple(fit.params)
    end

    function level_2026(fit)
        μ = fit.μ0 * exp(fit.β * x2026)
        quantile(GeneralizedExtremeValue(μ, fit.κ * μ, fit.ξ), 0.99)
    end

    trend_percent(fit) =
        100 * (exp(fit.β * (x2026 - x1950)) - 1)

    # 5. Weight functions to compare
    settings = [
        ("Linear, r=25 mi",
         d -> max(0.0, 1 - d / 25)),
```

```
        ("Linear, r=50 mi",
         d -> max(0.0, 1 - d / 50)),

        ("Region of influence, r=50 mi",
         d -> d <= 50 ? 1.0 : 0.0),

        ("Inverse squared, d0=10, r=50 mi",
         d -> d > 50 ? 0.0 : d <= 10 ? 1.0 : (10 / d)^2),

        ("Exponential, L=25 mi, no cutoff",
         d -> exp(-d / 25))
    ]

    results = DataFrame(
        setting = String[],
        n_stations = Int[],
        n_eff = Float64[],
        trend_percent = Float64[],
        level_2026_in = Float64[],
        lower_95_in = Float64[],
        upper_95_in = Float64[],
        successful_bootstraps = Int[]
    )

    # 6. Fit each setting and bootstrap over whole years
    for (label, weight_fn) in settings
        println("\nRunning: ", label)
        flush(stdout)

        regional = copy(data)
        regional.w = weight_fn.(regional.miles)
        filter!(r -> r.w > 0, regional)

        station_weights = unique(select(regional, :id, :w))
        fit = fit_model(regional.rainfall, regional.x, regional.w)

        years = sort(unique(regional.year))
        year_rows = [findall(==(yr), regional.year) for yr in years]

        boot_levels = Float64[]

        for b in 1:B
            selected = rand(1:length(years), length(years))
            rows = reduce(vcat, year_rows[selected])

            try
```

```
                boot_fit = fit_model(
                    regional.rainfall[rows],
                    regional.x[rows],
                    regional.w[rows]
                )

                level = level_2026(boot_fit)
                isfinite(level) || error("Nonfinite return level")
                push!(boot_levels, level)

            catch err
                err isa InterruptException && rethrow()
                @warn "Bootstrap fit failed" setting=label replicate=b exception=err
            end

            if b % 100 == 0
                println("  Bootstrap: $b / $B")
                flush(stdout)
            end
        end

        # Do not report an interval if too many fits failed
        adequate = length(boot_levels) >= ceil(Int, 0.95 * B)

        if length(boot_levels) < B
               @warn "Inspect failed fits before using this interval" setting=label
successful=length(boot_levels)
        end

        push!(results, (
            label,
            nrow(station_weights),
            sum(station_weights.w),
            trend_percent(fit),
            level_2026(fit),
            adequate ? quantile(boot_levels, 0.025) : NaN,
            adequate ? quantile(boot_levels, 0.975) : NaN,
            length(boot_levels)
        ))
    end

    # 7. Print final table
    for col in [
        :n_eff, :trend_percent, :level_2026_in,
        :lower_95_in, :upper_95_in
    ]
```

```
        results[!, col] = round.(results[!, col]; digits=2)
    end

    println("\nRainfall levels and bootstrap intervals are in inches.\n")
    show(stdout, results; allrows=true, allcols=true)
    println()
end
```

```
Running: Linear, r=25 mi
  Bootstrap: 100 / 500
  Bootstrap: 200 / 500
  Bootstrap: 300 / 500
  Bootstrap: 400 / 500
  Bootstrap: 500 / 500

Running: Linear, r=50 mi
  Bootstrap: 100 / 500
  Bootstrap: 200 / 500
  Bootstrap: 300 / 500
  Bootstrap: 400 / 500
  Bootstrap: 500 / 500

Running: Region of influence, r=50 mi
  Bootstrap: 100 / 500
  Bootstrap: 200 / 500
  Bootstrap: 300 / 500
  Bootstrap: 400 / 500
  Bootstrap: 500 / 500

Running: Inverse squared, d0=10, r=50 mi
  Bootstrap: 100 / 500
  Bootstrap: 200 / 500
  Bootstrap: 300 / 500
  Bootstrap: 400 / 500
  Bootstrap: 500 / 500

Running: Exponential, L=25 mi, no cutoff
  Bootstrap: 100 / 500
  Bootstrap: 200 / 500
  Bootstrap: 300 / 500
  Bootstrap: 400 / 500
  Bootstrap: 500 / 500

Rainfall levels and bootstrap intervals are in inches.
```

```
5×8 DataFrame
 Row │ setting                          n_stations  n_eff    trend_percent  level_2026_in
lower_95_in  upper_95_in  successful_bootstraps
     │ String                           Int64       Float64  Float64        Float64
Float64      Float64      Int64
─────┼──────────────────────────────────────────────────────────────────────────────────
   1 │ Linear, r=25 mi                          4     1.93         -3.83         13.68
10.31        18.12                    500
   2 │ Linear, r=50 mi                         20     6.13          9.83         16.24
12.92        20.93                    500
   3 │ Region of influence, r=50 mi            20    20.0          17.79         17.56
13.34        22.89                    500
   4 │ Inverse squared, d0=10, r=50 mi          20     3.45          3.76         15.19
11.94        19.52                    500
   5 │ Exponential, L=25 mi, no cutoff         816     9.68         16.28         16.61
12.75        22.15                    500
```

Share only the trend and shape

When every station shares every parameter, a wetter inland station pulls Scholes Field's location toward its own. `weighted_shared_trend` gives each station its own $\mu_{0,s}$ and κ_s and shares only β and ξ:

$$y_{s,t} \sim \mathrm{GEV}(\mu_{s,t}, \kappa_s \mu_{s,t}, \xi), \qquad \mu_{s,t} = \mu_{0,s} e^{\beta x_{s,t}}.$$

```
@model function weighted_shared_trend(y, x, station, w, n_stations)
    μ0 ~ filldist(LogNormal(1.5, 0.5), n_stations)
    κ ~ filldist(LogNormal(-1, 0.5), n_stations)
    β ~ Normal(0, 0.5)
    ξ ~ Uniform(-0.5, 0.5)
    μ = μ0[station] .* exp.(β .* x)
    σ = κ[station] .* μ
    if !all(σ .> 0)
        Turing.@addlogprob! -Inf
        return nothing
    end
    Turing.@addlogprob! sum(w .* logpdf.(GeneralizedExtremeValue.(μ, σ, ξ), y))
end
```

We fit it the same way as `weighted_gevfit`, starting every station from the same values, and keep Scholes Field's parameters.

```
y = ann_max_region.prcp_in
n_stations = nrow(stations)
station = [findfirst(==(id), stations.id) for id in ann_max_region.id]
start = InitFromParams((μ0=fill(median(y), n_stations),
```

```
                        κ=fill(std(y) / median(y), n_stations), β=0.0, ξ=0.1))
model = weighted_shared_trend(y, ann_max_region.x, station, ann_max_region.w,
                              n_stations)
shared_fit = maximum_a_posteriori(model; initial_params=start)
if string(shared_fit.optim_result.retcode) != "Success"
    @warn "shared-trend fit did not converge" shared_fit.optim_result.retcode
end
(; μ0, κ, β, ξ) = NamedTuple(shared_fit.params)
k = findfirst(==(scholes_field.id), stations.id)
shared_trend = (μ0=μ0[k], κ=κ[k], β=β, ξ=ξ)

for (label, fit) in ("all shared" => pooled,
                     "trend and shape shared" => shared_trend)
    @printf("%-22s μ0 = %.2f, κ = %.2f, trend %+.0f%%, ξ = %.2f, 2026 100-year level
%.1f inches\n",
            label, fit.μ0, fit.κ, trend_percent(fit), fit.ξ, level_2026(fit))
end
```

Line 3

`station[i]` is the row of `stations`, from 1 to `n_stations`, that `y[i]` came from.

Line 13

Scholes Field's row of `stations`.

```
all shared             μ0 = 3.46, κ = 0.40, trend +10%, ξ = 0.22, 2026 100-year level
16.2 inches
trend and shape shared μ0 = 3.52, κ = 0.41, trend +3%, ξ = 0.24, 2026 100-year level
16.2 inches
```

Your answer: Which parameters should be shared for Scholes Field, and why? I would share the trend β and shape ξ, while keeping μ_0 and κ station-specific to preserve local rainfall differences.

Defend a choice

Your answer: What is your estimate of the 100-year daily rainfall at Galveston Scholes Field in 2026, with an interval? Say which weight function and radius you chose, which parameters you shared, and which interval you report, and defend each choice.

My estimate of the 2026 100-year daily rainfall at Galveston Scholes Field is 15.48 inches, with a 95% year-bootstrap interval of 10.45–21.12 inches. I chose inverse-distance-squared weights with $d_0 = 10$ miles and a 50-mile radius. This gives nearby stations more influence while reducing contributions from distant stations. I chose inverse-distance-squared weights because, from a geographical perspective, I expect rainfall similarity to decrease rapidly with distance.With further investigation, weights could incorporate indicators of climate similarity, rather than distance alone, to better identify stations representative of Galveston Island's rainfall conditions. I shared the trend parameter β and shape parameter ξ because Scholes Field's short record provides limited information about the long-term trend and tail shape. Borrowing information from nearby stations may improve their estimation, while station-specific μ_0 and

κ preserve local rainfall differences. I report the year-bootstrap interval because it keeps each year's observations together, preserving dependence across stations during shared storm events.

```
let
    Random.seed!(543)

    radius = 50.0
    d0 = 10.0
    B = 500
    target_id = "79-0047"

    # 1. Inverse-distance-squared weights
    weight_fn(d) = d > radius ? 0.0 :
                   d <= d0 ? 1.0 : (d0 / d)^2

    data = copy(ann_max_texas)
    data.w = weight_fn.(data.miles)
    filter!(row -> row.w > 0, data)

    # 2. Assign each station an integer index
    station_ids = unique(data.id)
    n_stations = length(station_ids)

    id_to_index = Dict(id => i for (i, id) in enumerate(station_ids))
    data.station_index = [id_to_index[id] for id in data.id]
    target_index = id_to_index[target_id]

    station_weights = unique(select(data, :id, :w))

    # 3. Fit the shared-trend-and-shape model
    function fit_shared(rows)
        y = rows.prcp_in

        start = InitFromParams((
            μ0 = fill(median(y), n_stations),
            κ = fill(std(y) / median(y), n_stations),
            β = 0.0,
            ξ = 0.1
        ))

        model = weighted_shared_trend(
            y, rows.x, rows.station_index, rows.w, n_stations
        )

        fit = maximum_a_posteriori(model; initial_params=start)
```

```
        if string(fit.optim_result.retcode) != "Success"
            error("Fit did not converge: $(fit.optim_result.retcode)")
        end

        params = NamedTuple(fit.params)

        # Return Scholes Field's parameters
        (
            μ0 = params.μ0[target_index],
            κ = params.κ[target_index],
            β = params.β,
            ξ = params.ξ
        )
    end

    target_fit = fit_shared(data)

    # 4. Bootstrap whole years, preserving station indices
    years = sort(unique(data.year))
    year_rows = [findall(==(yr), data.year) for yr in years]

    bootstrap_levels = Float64[]

    for b in 1:B
        picked = rand(1:length(years), length(years))
        row_indices = reduce(vcat, year_rows[picked])
        resampled = data[row_indices, :]

        # Stop on failure rather than silently omit a replicate
        boot_fit = try
            fit_shared(resampled)
        catch err
            println("Bootstrap replicate $b failed. Inspect the error before reporting
an interval.")
            rethrow()
        end

        level = level_2026(boot_fit)
        isfinite(level) || error("Nonfinite level at replicate $b")
        push!(bootstrap_levels, level)

        if b % 50 == 0
            println("Bootstrap: $b / $B")
            flush(stdout)
        end
    end
```

```
    # 5. Print results
    lower = quantile(bootstrap_levels, 0.025)
    upper = quantile(bootstrap_levels, 0.975)

    println("\nInverse-distance-squared weights: d0=10 mi, radius=50 mi")
    println("Shared parameters: β and ξ")
    @printf("Stations kept: %d\n", n_stations)
    @printf("n_eff: %.2f\n", sum(station_weights.w))
    @printf("Scholes Field μ0: %.2f\n", target_fit.μ0)
    @printf("Scholes Field κ: %.2f\n", target_fit.κ)
    @printf("Shared β: %.3f\n", target_fit.β)
    @printf("Shared ξ: %.3f\n", target_fit.ξ)
    @printf("Trend, 1950–2026: %+.2f%%\n", trend_percent(target_fit))
    @printf("2026 100-year rainfall: %.2f inches\n", level_2026(target_fit))
    @printf("95%% year-bootstrap interval: %.2f–%.2f inches\n", lower, upper)
end
```

```
Bootstrap: 50 / 500
Bootstrap: 100 / 500
Bootstrap: 150 / 500
Bootstrap: 200 / 500
Bootstrap: 250 / 500
Bootstrap: 300 / 500
Bootstrap: 350 / 500
Bootstrap: 400 / 500
Bootstrap: 450 / 500
Bootstrap: 500 / 500

Inverse-distance-squared weights: d0=10 mi, radius=50 mi
Shared parameters: β and ξ
Stations kept: 20
n_eff: 3.45
Scholes Field μ0: 3.66
Scholes Field κ: 0.41
Shared β: -0.015
Shared ξ: 0.222
Trend, 1950–2026: -0.68%
2026 100-year rainfall: 15.48 inches
95% year-bootstrap interval: 10.45–21.12 inches
```

Turning it in

This lab is due on Canvas on Friday, October 16, the same day as Test 1.

1. Check that your notebook answers each **Your answer** question.

2. Render the notebook to a PDF. In the terminal, from the lab folder, run:

   ```
   quarto render index.qmd --to typst
   ```

   This writes `index.pdf` beside the notebook. It runs the whole notebook from the top in a fresh process, so it also confirms your work does not depend on leftover REPL state.

3. Commit and push your work, with GitHub Desktop or by asking Claude to commit and push.

4. Upload `index.pdf` to the Lab 7 assignment on Canvas.

5. Paste the link to your repository in the same Canvas submission.

If the render fails and you cannot fix it, upload what you have and say where it broke. A lab that does not render is worth more to me than a lab you did not turn in.

Bibliography

[1] S. Perica, S. Pavlovic, M. St. Laurent, C. Trypaluk, D. Unruh, and O. Wilhite, "NOAA Atlas 14," Silver Spring, MD, Volume 11 Version 2.0: Texas, 2018. Accessed: Sep. 22, 2021. [Online]. Available: https://www.weather.gov/media/owp/oh/hdsc/docs/Atlas14_Volume11.pdf

[2] D. M. Etheridge, L. P. Steele, R. L. Langenfelds, R. J. Francey, J. Barnola, and V. I. Morgan, "Natural and Anthropogenic Changes in Atmospheric CO\textsubscript{2} over the Last 1000 Years from Air in Antarctic Ice and Firn," *Journal of Geophysical Research: Atmospheres*, vol. 101, no. D2, pp. 4115–4128, Feb. 1996, doi: 10.1029/95jd03410.

[3] C. MacFarling Meure *et al.*, "Law Dome CO\textsubscript{2}, CH\textsubscript{4} and N\textsubscript{2}O Ice Core Records Extended to 2000 Years BP," *Geophysical Research Letters*, vol. 33, no. 14, Jul. 2006, doi: 10.1029/2006gl026152.